

Development of an Integrated DataOps and MLOps Platform Architecture on Kubernetes Using Open Source Tools

Bryan P. Hutagalung

*Information Systems and Technology
School of Electrical Engineering and Informatics
Institut Teknologi Bandung
Bandung, Indonesia
18222130@std.stei.itb.ac.id*

Abstract—DataOps and MLOps are commonly adopted side by side, yet the tools supporting them remain fragmented and the two disciplines are usually operated by separate teams. A team that wants both must either assemble the integration itself, which takes time, or subscribe to managed services, which carries recurring cost and ties the platform to one provider. This paper develops an integrated DataOps and MLOps platform architecture on Kubernetes built from open source tools. Design Science Research Methodology guides the work from problem identification through design, GitOps-based implementation, and evaluation. Nine platform layers are derived from the convergence of five sources of differing method rather than from majority adoption, and components for each layer are selected through scored comparison against four criteria. The design joins the two disciplines at three points, namely a shared feature contract, an open lineage protocol feeding one metadata catalog, and one declarative control plane reconciled from a single Git repository. A crypto asset market use case verifies the platform on a single-node cluster. Evaluation covers 26 requirements, of which two are verified functionally, five are instrumented with their target left unmeasured, and 19 are verified structurally. The retraining chain fires when injected drift crosses the configured threshold, yielding a Population Stability Index of 18.107 against a threshold of 0.2. A cost comparison across three provisioning options places the open source design below fully managed services over three years under an assumed labour allocation. Load characterisation under real traffic remains open, on one node and across a cluster.

Index Terms—DataOps, MLOps, Kubernetes, feature store, drift detection, GitOps, open source

I. INTRODUCTION

DataOps improves the quality and speed of the data flow, while MLOps improves the reliability of the model lifecycle. Teams generally need both. In practice the two grow apart, because the model code is only a small part of a production machine learning system and most of the surrounding work is data management, configuration, monitoring, and serving [1]. When the two disciplines are operated separately, lineage, drift detection, and feature serving end up inconsistent across the cycle.

Building the combined platform in-house runs into the breadth of the choice space. A systematic mapping of 43

primary studies identifies 35 recurring MLOps architecture components [2], and open source components in this space tend to stand alone rather than form a cohesive framework [3]. Integration work, learning curve, and version alignment therefore fall on the adopting team from the start.

The alternative is to subscribe to managed services, which reduces configuration work but introduces recurring cost that is hard to estimate in advance and can become uneconomical as load grows [4]. Organisations are left choosing between a long build and a subscription with limited room for customisation.

Neither choice removes the integration work between the ingestion, feature, training, and serving layers. Integration is then reassembled per project, and metadata, identity, and lineage are managed separately, so provenance is hard to trace and feature values can differ between training and serving. Surveys of deployment practice report pipeline maintenance and inter-component integration as recurring difficulties [5].

This paper develops an integrated DataOps and MLOps platform architecture on Kubernetes built from open source tools. The architecture combines four properties that prior work has addressed separately, namely DataOps and MLOps unified on one Kubernetes control plane, data governance enforced automatically along the production pipeline, drift detection that triggers retraining through that same declarative control, and a dual-store feature service under one contract with vector search alongside it on a separate index. No individual element is new. The contribution is the combination, which the designs reviewed in Section II do not cover together.

II. RELATED WORK

MLOps is defined as a combination of working culture and software that automates experimentation, training, evaluation, deployment, and monitoring [6]. That definition frames what an integrated platform has to cover on the model side, and DataOps adds ingestion, quality, and governance on the data side [7].

Four studies sit close to this work. Burgueño-Romero et al. propose an open source MLOps architecture assembled from components, but a feature service with vector support

is not among them [3]. Amou Najafabadi et al. map MLOps architectures across 43 primary studies, but the mapping does not combine DataOps comprehensively [2]. Drift detection and model updating are the emphasis of Rodrigues et al., whose architecture targets near real-time distributed stream learning, though data governance is not enforced as a control plane there [8]. Ahmad et al. discuss MLOps-enabled security strategies without linking them to a governance control plane that enforces access policy across the data and model lifecycle [9].

Because those four properties are handled separately, feature lineage from ingestion through to model serving is hard to trace on one control plane.

A second gap is more specific. Amou Najafabadi et al. observe that an explicit mapping between concrete tools and architecture components is still missing from the MLOps literature [2]. Section IV supplies that mapping for the open source case, with the scoring shown.

III. METHOD

The work follows Design Science Research Methodology [10]. The main output is a running platform, which is an instantiation artifact in the design science taxonomy [11], so an artifact-oriented framework fits better than a pure software engineering process.

The architecture targets MLOps level 2 on the Google Cloud maturity scale [12], whose seven required components the layers of Section IV-A cover. Practice that keeps the two disciplines apart, as described in Section I, sits at level 0 on the same scale. Level 2 is what demands the closed CI, CD, and continuous training loop that Sections IV-C and IV-D assemble. Whether that loop performs at level 2 under production load is not claimed here.

The six phases run iteratively. Problem identification produces four problem statements from a study of DataOps, MLOps, and data governance. Objective definition turns each statement into an artifact that can be installed, operated, and audited, giving four objectives mapped one to one onto the problem statements.

- **T-1** unifies ingestion, processing, storage, feature store, training, serving, governance, and observability into one declarative control plane with GitOps practice.
- **T-2** builds a data governance sub-system providing meta-data catalog, lineage recording, and quality control as shared services along the pipeline.
- **T-3** implements automated drift detection with a threshold-based retraining policy, and enforces point-in-time correctness across batch and streaming modes through the historical feature join.
- **T-4** develops a dual-store feature service delivering tabular and aggregate features under one API contract, with vector embeddings served on a separate index whose embedding space stays consistent between training and inference.

The objectives are decomposed into 14 functional and 12 non-functional requirements, which become the units of

evaluation in Section VI. Design and development classify the services named by the sources in Section IV-A into platform layers, select open source components per layer, and express the result as a declarative control plane. Before that, four provisioning alternatives were scored against the same 26 requirements from vendor documentation and public offerings, and open source on self-managed Kubernetes scored highest, mainly on temporal validity, vector support, portability, and extensibility. That scoring is desk research preceding implementation and is not an evaluation result. It is also separate from the three-way cost comparison in Section VI-C, which prices only the options that remained practical. Demonstration installs the platform on k3s and drives it with one real domain use case. Evaluation examines functional fulfilment, non-functional targets, and governance, and communication produces this paper alongside the source repository.

IV. ARCHITECTURE

A. Deriving the Platform Layers

The layers are derived from convergence across sources that used different methods, not from majority adoption. On the model side, Kreuzberger et al. name nine MLOps components through mixed-method research combining literature review, tool review, and expert interviews [6]. Amershi et al. arrive at a nine-stage workflow by observing engineering teams at Microsoft, and already separate the data-oriented stages from the model-oriented ones [13]. Starting from different methods, the two name largely the same components. Amou Najafabadi et al. synthesise 35 components from 43 primary studies and report how often each occurs, with model registry at 21 occurrences, ML metadata repository at 15, and feature store at 8 [2]. Google Cloud lists seven components required for MLOps level 2 [12].

The occurrence counts carry less weight than they suggest, because no single component reaches an absolute majority of the 43 studies. What they show is that the same components recur across independent studies.

On the data side, Munappy et al. derive an evolution model from ad-hoc analytics to DataOps from an industrial case study, naming data collection, data testing, and data monitoring as recurring practices, and treating controlled access and data-integrity protection as governance [7]. The data field therefore widens the MLOps set rather than duplicating it. It is the convergence across these five sources of differing method that forms the evidence base, not a claim that any one component is used by most of the industry.

Consolidating both sides gives the nine layers in Table I. Four cover the data domain and five cover the model domain and platform operations. Each row traces to the sources that name its services, and the backing is not even across rows. The Model Lifecycle layer is named by four sources at once, whereas the Security layer draws mainly on the security governance dimension of the DataOps side. Table I leaves that difference visible instead of levelling it.

Two boundaries are deliberate. Monitoring is placed in Observability rather than in Serving, so that measurement does

TABLE I
PLATFORM LAYERS, THEIR EVIDENCE BASE, AND THE COMPONENTS SELECTED FOR EACH

Layer	Recurring services named by the sources	Sources	Selected open source components
Orchestration and Infrastructure	model training infrastructure, infrastructure and supporting services	[6], [2], [12]	k3s, HPA, VPA recommender, KEDA, Kueue, cert-manager
Data Ingestion and Processing	data collection, data cleaning, data preprocessor, data ingestion	[13], [2], [7]	Kafka with Strimzi, Karapace, Debezium, Flink, Spark, dbt, Airflow, Trino
Data Storage	dataset repository, data storage, data versioning	[2], [7]	MinIO, Apache Iceberg, Lakekeeper, lakeFS, ClickHouse, PostgreSQL, MySQL, OpenSearch
Feature Service	feature store, offline and online serving	[6], [12], [2]	Feast, ClickHouse offline, Valkey online, Qdrant on a separate index outside the Feast contract
Model Lifecycle	model training, model evaluation, model registry, ML metadata store, inference service	[6], [13], [12], [2]	Kubeflow Pipelines, Katib, Kubeflow Trainer, MLflow, KServe, Knative, Argo Workflows
Data Governance	data testing, controlled access, privacy and data integrity protection	[7]	DataHub, OpenLineage, Great Expectations
Observability	monitoring component, model monitoring, data monitoring	[6], [13], [2], [7]	Prometheus, Loki, Grafana Tempo, Grafana, OpenTelemetry, Superset, Evidently, OpenCost
GitOps and Continuous Delivery	CI/CD component, source code repository, deployment services	[6], [12], [2]	Argo CD, Gitea, Tekton, Argo Rollouts
Security	controlled access, privacy and data security compliance, API gateway	[7], [2]	Istio, APISIX, dex, oauth2-proxy, SpiceDB, OpenBao, Kyverno, Falco

not mix with inference. Drift detection is not a tenth layer. It is a mechanism that crosses layers, connecting the monitoring function to the model lifecycle, and it is realised as a sub-system instead.

The layer order follows the concept order of the underlying literature study one to one, so the partitioning is not a new taxonomy standing on its own.

B. Selecting Components

Every role with more than one viable candidate is scored in a comparison matrix. Each matrix scores four criteria chosen for that role, so the set is not fixed across matrices. Maturity, community size, and Kubernetes integration recur most often, while the rest are role-specific, such as batch and streaming capability for processing engines or footprint for Kubernetes distributions. Each criterion scores 0, 1, or 2, giving a maximum total of 8. A score of 0 marks a criterion that is not met, 1 marks partial support that still needs an additional component or carries a significant limitation, and 2 marks full support by built-in capability. Scores are derived from official documentation and each project’s maintenance status. Twenty-four matrices cover the roles across the nine layers, and their winners appear in the last column of Table I alongside the components adopted without a contest. Nineteen of the 24 winners take a perfect 8.

The highest total takes the role, but it does not remove the other candidates. A tool with a lower total can still be adopted where it is stronger on the criterion that matters. Spark takes the highest total and the large-scale batch role, while Flink, which scores lower overall but higher on event-time processing with exactly-once checkpointing, takes the streaming role. Neither displaces the other. Kubeflow Pipelines takes the training flow, Argo Workflows runs the scheduled control loop, and Airflow orchestrates the batch data pipeline. The three roles do not overlap.

C. Joining the Two Disciplines

Fig. 1 shows the nine layers as one architecture. DataOps and MLOps meet at three points, and at each one the producing and the consuming side read the same definition rather than two copies of it.

The first is the feature contract. Feast binds the offline store and the online store to one feature definition, so the pipelines that write features and the training and inference code that read them use the same contract.

The second is open lineage. OpenLineage carries transformation events from both the data orchestrator and the training orchestrator, including the SQL transformation steps inside them, into one metadata catalog. The lineage of a training dataset is then traceable from the same graph that holds the ingestion assets.

The third is the declarative control plane. Argo CD reconciles the manifests of both disciplines from one Git repository, so policy and component versions stay consistent across the two. It is also what makes the drift loop of Section IV-D declarative.

D. Drift Detection and Retraining

Drift detection sits where the two disciplines meet, because drift is observed on the data layer while its consequence is executed on the model layer. A detector runs as a scheduled job, every 15 minutes for volatile features and hourly for more stable ones. It reads a current window and a reference window from ClickHouse and computes the Population Stability Index with adaptive binning and the Kolmogorov-Smirnov statistic with per-feature thresholds [14]. Scores are written back to an analytics table in ClickHouse for audit, and pushed to Prometheus for alerting.

A scheduled Argo CronWorkflow reads those scores and decides whether to trigger training. Thresholds live in a Git-committed ConfigMap, and each run records a checksum of the configuration version, so a policy change is auditable

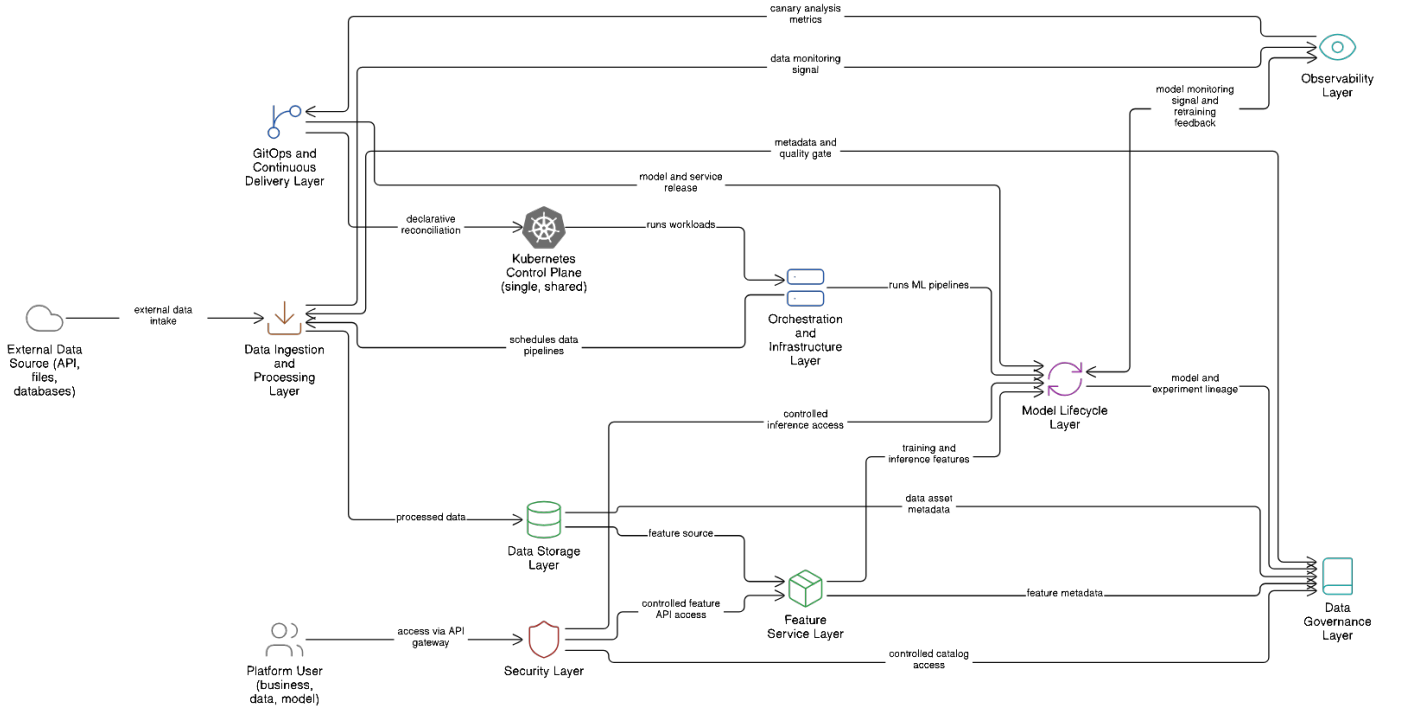


Fig. 1. Integrated service architecture on a shared Kubernetes control plane, drawn without binding to any tool [12]

without inspecting the cluster. When a threshold is crossed, the workflow calls the training pipeline, which assembles its training set through a point-in-time join and logs the run to the model registry as a new version.

Whether that candidate replaces the running model is decided at canary promotion on live serving metrics, not by a separate comparison before deployment. Promotion is configured to halt and roll back when a metric crosses its threshold.

E. Dual-Store Feature Service

The dual-store design is two stores under one contract. ClickHouse holds offline features for training, Valkey serves online lookups at inference, and both derive their schema from the same Feast definition. Vector embeddings are handled separately. Qdrant stands outside Feast as a vector index reached through its native client, because an approximate nearest neighbour load needs memory for the HNSW graph [15] and behaves differently from a key-value point lookup. Separating them lets each be tuned on its own.

Point-in-time correctness is enforced through the historical feature join, which admits only values whose event timestamp does not exceed each entity row’s reference timestamp and still falls inside the feature view’s time-to-live. Training therefore sees the values that would have been available at that instant, which is what prevents the optimistic evaluation that leakage produces [16]. Batch and stream pipelines write to stores registered on the same feature view, so a consumer does not need to know which pipeline produced a value.

V. IMPLEMENTATION

The platform runs on k3s on a single node [17]. The node count is an implementation choice rather than part of the contribution. Namespace layout is standardised so that moving to a multi-node cluster should change only the Kustomize overlay, leaving the base manifests untouched. That move has not been performed. Library versions and base images are pinned to the highest stable release as of April 2026 so that testing is reproducible, and a component is intended to be replaceable by one of equivalent capability without changing the design.

Components are grouped one namespace per layer and joined by stable API contracts, which lets network policy be enforced default-deny with explicitly declared exceptions. Argo CD is the only path by which change reaches the cluster [18], with every component rendered from an ApplicationSet template so that adding a component adds no boilerplate. Tekton builds and tests. Istio enforces mutual TLS in strict mode on internal traffic, Kyverno validates at admission, and Falco watches the kernel at runtime. One idle replica per workload is the default, with autoscaling templates prepared but not exercised under load.

Platform code stays domain-agnostic in its own directory. The domain lives in a separate use-case directory applied through a Kustomize overlay that sets only the name prefix, registry mapping, initial replicas, and autoscaling bounds. Adding another use case should require only copying and adjusting domain configuration, without touching platform code, though this has not been tried.

TABLE II
VERIFICATION OUTCOME PER OBJECTIVE

Objective	Req.	Func.	Instr.	Struct.
T-1	12	0	2	10
T-2	6	1	0	5
T-3	3	1	0	2
T-4	5	0	3	2
Total	26	2	5	19

The verification use case is crypto asset market analytics on Coinbase data, chosen for three reasons. It exercises all nine layers at once, so verification runs against the full service set rather than a subset. Crypto markets trade continuously with no closing hours [19], so the ingestion pipeline and stream processing are driven by real event flow around the clock rather than by synthetic load on a schedule. Price volatility is high and clustered [20], so feature distributions are expected to shift within hours, which is what makes the domain suitable for drift work. The drift result in Section VI-B nevertheless comes from a controlled injection, not from natural shift. Two ingestion pipelines run. A websocket collector feeds Flink for windowed features, and a REST collector backfills one-minute candles from 1 March 2026.

VI. EVALUATION

A. What Each Claim Rests On

Claims are reported at three tiers, each naming what was actually observed. A requirement is *structurally verified* when the control plane is formed and reconciled as designed. It is *functionally verified* when a test scenario ran and its output met the acceptance criterion. It is *instrumented* when the measuring instrument is installed and the flow is serving requests, while the quantitative target itself was not measured.

Every scenario was run manually, outside the reconciliation path, and the load generator ran from a separate namespace, so that measurement did not disturb the pipeline under test. Table II gives the distribution.

B. Results

The integrated architecture and its GitOps path (T-1) were verified structurally. All components install and reconcile from one Git repository, definition changes converge without drift, and the whole platform was rebuilt on a fresh k3s node with no manual step beyond the initial bootstrap. That reconstruction is the strongest evidence for the declarative claim, because it exercises the entire manifest set rather than a sample.

Governance (T-2) contributed one of the two functional results. The quality gate rejects schema and range violations before the batch moves downstream. Lineage events from the data and training orchestrators, together with catalog ingestion from the feature store, model registry, warehouse, and broker, assemble into one graph holding 270 datasets, 18 data jobs, and 3 data flows. The graph measures what the emitters covered, not completeness across every asset.

Drift detection (T-3) contributed the other. A controlled 2-sigma injection on one feature produced a Population Stability Index of 18.107 against a 0.2 threshold and a Kolmogorov-Smirnov statistic of 0.815 against a 0.15 threshold, and the scheduled workflow triggered one retraining run. This shows the chain from detection to trigger fires. It does not show detection accuracy against natural distribution shift over time, which needs longer observation on real streams.

The feature service (T-4) is the weakest of the four. The online, offline, and vector flows are formed and serving requests, but three of its five requirements are instrumented rather than measured. Vector search returned a median of 4.25 ms and a 95th percentile of 5.02 ms over 100 iterations, though the test collection was empty, so those figures describe an installed and responsive flow rather than retrieval quality. Recall was not tested. Equality of feature values between the online and offline stores holds by construction, since materialisation copies from one to the other, but the per-entity comparison that would confirm it is still a manual procedure. Batch and stream consistency was established at the level of feature definitions, not values.

C. Cost

Three provisioning options were priced against a reference capacity of 16 vCPU, 64 GB memory, and roughly 400 GB storage. Infrastructure figures are public list prices retrieved in June 2026, with two providers priced in July where the June archive carried no figure. Over three years, fully managed cloud services reach USD 39,543.16 and building every layer from scratch on self-managed infrastructure reaches USD 45,461.16, against USD 23,239.22 for the open source design used here.

The labour allocation behind those totals is an illustrative assumption rather than a measurement. The open source option is credited with dropping to a quarter of a full-time engineer after the first year on the argument that reconciliation, unified monitoring, and automated retraining absorb daily operations. The figures therefore show which option is cheaper under that assumption, and nothing more. They are not a measurement of what this platform cost to run.

D. Threats to Validity

The test environment is a single node, so horizontal scalability was exercised through multiple replicas and autoscaling on one machine rather than across a cluster. Near-linear scaling was not tested. No p99 under load, peak throughput, or saturation point was measured. Drift rests on one controlled injection on one feature. Every number comes from a single run, with no repetitions and no variance reported. The platform was not benchmarked against a managed stack or any alternative, so the cost comparison contrasts hypothetical provisioning options rather than measured systems. Domain independence is argued from the separation between platform and use-case code and has not been demonstrated on a second use case.

VII. CONCLUSION

The architecture developed here puts DataOps and MLOps on one Kubernetes control plane using open source tools throughout. Nine layers were derived from convergence across sources of differing method, components were selected through scored comparison, and the two disciplines were joined at the three points of Section IV-C. The value of the result is that integration effort previously repeated per project is consolidated into a single declarative description, so rebuilding the control plane no longer depends on one engineer's undocumented knowledge.

The quality gate rejecting bad batches and the automatic firing of the retraining chain are the two results that met an acceptance criterion. The rebuild of the whole platform from one Git source is the strongest of the structural results. Most requirements sit at that structural tier, which establishes that the control plane forms as designed and not that it performs at production load.

The most immediate next step is load characterisation under real traffic, on one node and then across a cluster, because the platform's behaviour under load is currently unknown. Drift observation needs longer runs on real streams before detection accuracy can be assessed. The per-entity comparison of online against offline feature values should be automated into the evaluation flow, so that the equality currently argued from construction is confirmed by measurement.

REFERENCES

- [1] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, Eds. Red Hook, NY: Curran Associates, Inc., 2015, pp. 2503–2511. [Online]. Available: <https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>
- [2] F. Amou Najafabadi, J. Bogner, I. Gerostathopoulos, and P. Lago, "An analysis of mlops architectures: A systematic mapping study," in *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, 3–6 September 2024, Proceedings*, ser. Lecture Notes in Computer Science, vol. 14889. Springer, 2024, pp. 69–85. [Online]. Available: https://doi.org/10.1007/978-3-031-70797-1_5
- [3] A. M. Burgueño-Romero, A. Benítez-Hidalgo, C. Barba-González, and J. F. Aldana-Montes, "Toward an open source mlops architecture," *IEEE Software*, vol. 42, no. 1, pp. 59–64, 2025. [Online]. Available: <https://doi.org/10.1109/MS.2024.3421675>
- [4] M. Hamza, M. A. Akbar, and R. Capilla, "Understanding cost dynamics of serverless computing: An empirical study," in *Software Business: 14th International Conference, ICSOB 2023, Proceedings*, ser. Lecture Notes in Business Information Processing, vol. 500. Springer, 2024, pp. 456–470. [Online]. Available: https://doi.org/10.1007/978-3-031-53227-6_32
- [5] A. Paleyes, R.-G. Urma, and N. D. Lawrence, "Challenges in deploying machine learning: A survey of case studies," *ACM Computing Surveys*, vol. 55, no. 6, pp. 1–29, 2022. [Online]. Available: <https://doi.org/10.1145/3533378>
- [6] D. Kreuzberger, N. Kühl, and S. Hirschl, "Machine learning operations (mlops): Overview, definition, and architecture," *IEEE Access*, vol. 11, pp. 31 866–31 879, 2023. [Online]. Available: <https://doi.org/10.1109/ACCESS.2023.3262138>
- [7] A. R. Munappy, D. I. Mattos, J. Bosch, H. H. Olsson, and A. Dakkak, "From ad-hoc data analytics to dataops," in *Proceedings of the International Conference on Software and System Processes (ICSSP '20)*. ACM, 2020, pp. 165–174. [Online]. Available: <https://doi.org/10.1145/3379177.3388909>
- [8] M. G. Rodrigues, E. K. Viegas, A. O. Santin, and F. Enembreck, "A mlops architecture for near real-time distributed stream learning operation deployment," *Journal of Network and Computer Applications*, vol. 238, p. 104169, 2025. [Online]. Available: <https://doi.org/10.1016/j.jnca.2025.104169>
- [9] T. Ahmad, M. Adnan, S. Rafi, M. A. Akbar, and A. Anwar, "Mlops-enabled security strategies for next-generation operational technologies," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024)*, 2024, pp. 662–670. [Online]. Available: <https://doi.org/10.1145/3661167.3661283>
- [10] K. Peffers, T. Tuunanen, M. A. Rothenberger, and S. Chatterjee, "A design science research methodology for information systems research," *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007. [Online]. Available: <https://doi.org/10.2753/MIS0742-1222240302>
- [11] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004. [Online]. Available: <https://doi.org/10.2307/25148625>
- [12] Google Cloud, "Mlops: Continuous delivery and automation pipelines in machine learning," Cloud Architecture Center, 2024, accessed: Mar. 18, 2026. [Online]. Available: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>
- [13] S. Amershi, A. Begel, C. Bird, R. DeLine, H. Gall, E. Kamar, N. Nagappan, B. Nushi, and T. Zimmermann, "Software engineering for machine learning: A case study," in *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*. IEEE, 2019, pp. 291–300. [Online]. Available: <https://doi.org/10.1109/ICSE-SEIP.2019.00042>
- [14] D. M. d. Reis, P. Flach, S. Matwin, and G. Batista, "Fast unsupervised online drift detection using incremental kolmogorov-smirnov test," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. ACM, 2016, pp. 1545–1554. [Online]. Available: <https://doi.org/10.1145/2939672.2939836>
- [15] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020. [Online]. Available: <https://doi.org/10.1109/TPAMI.2018.2889473>
- [16] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.
- [17] Cloud Native Computing Foundation, "Kubernetes documentation," 2024, accessed: Mar. 11, 2026. [Online]. Available: <https://kubernetes.io/docs/>
- [18] T. Limoncelli, "Gitops: A path to more self-service it," *Communications of the ACM*, vol. 61, no. 9, pp. 38–42, 2018. [Online]. Available: <https://dl.acm.org/doi/10.1145/3233241>
- [19] I. Makarov and A. Schoar, "Trading and arbitrage in cryptocurrency markets," *Journal of Financial Economics*, vol. 135, no. 2, pp. 293–319, 2020. [Online]. Available: <https://doi.org/10.1016/j.jfineco.2019.07.001>
- [20] P. Katsiampa, "Volatility estimation for bitcoin: A comparison of garch models," *Economics Letters*, vol. 158, pp. 3–6, 2017. [Online]. Available: <https://doi.org/10.1016/j.econlet.2017.06.023>